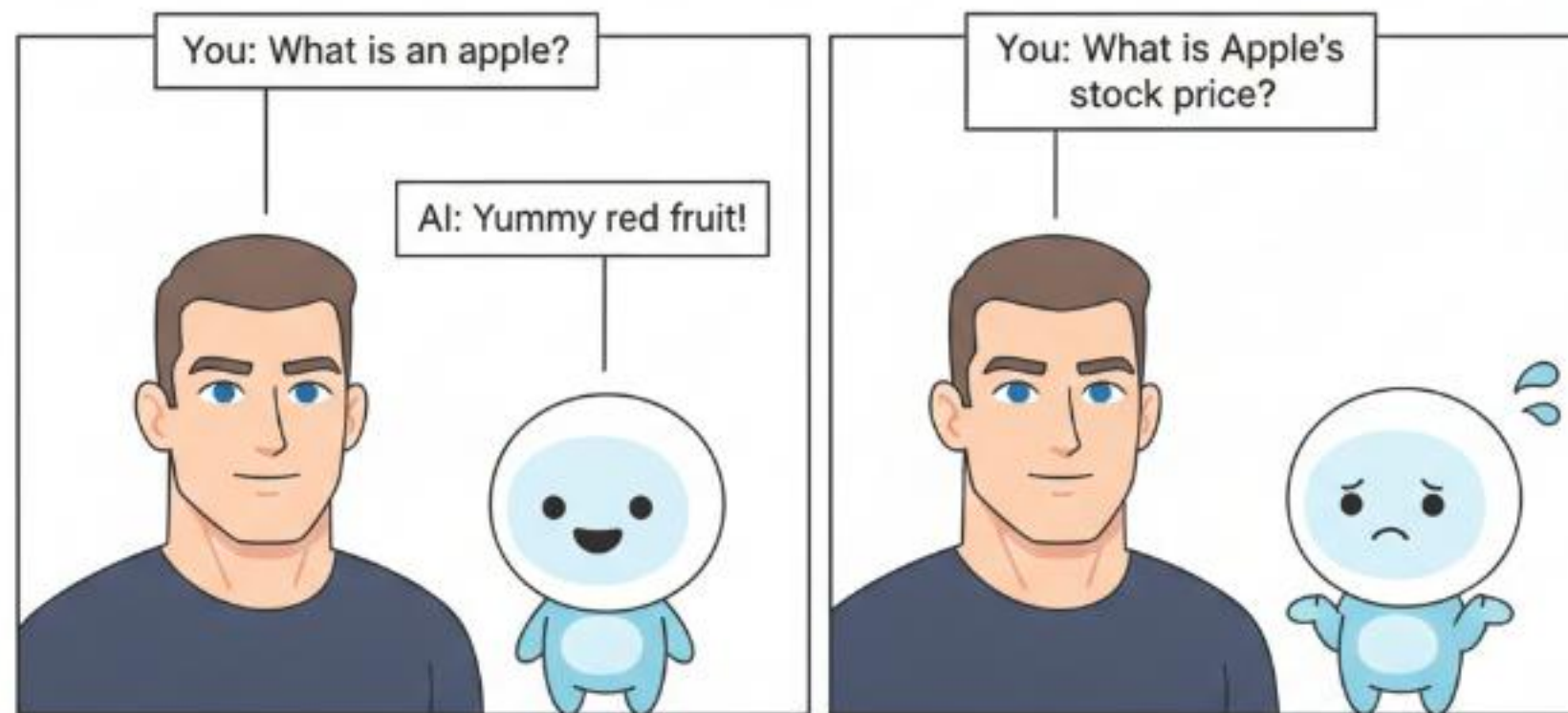


Tools & Skills



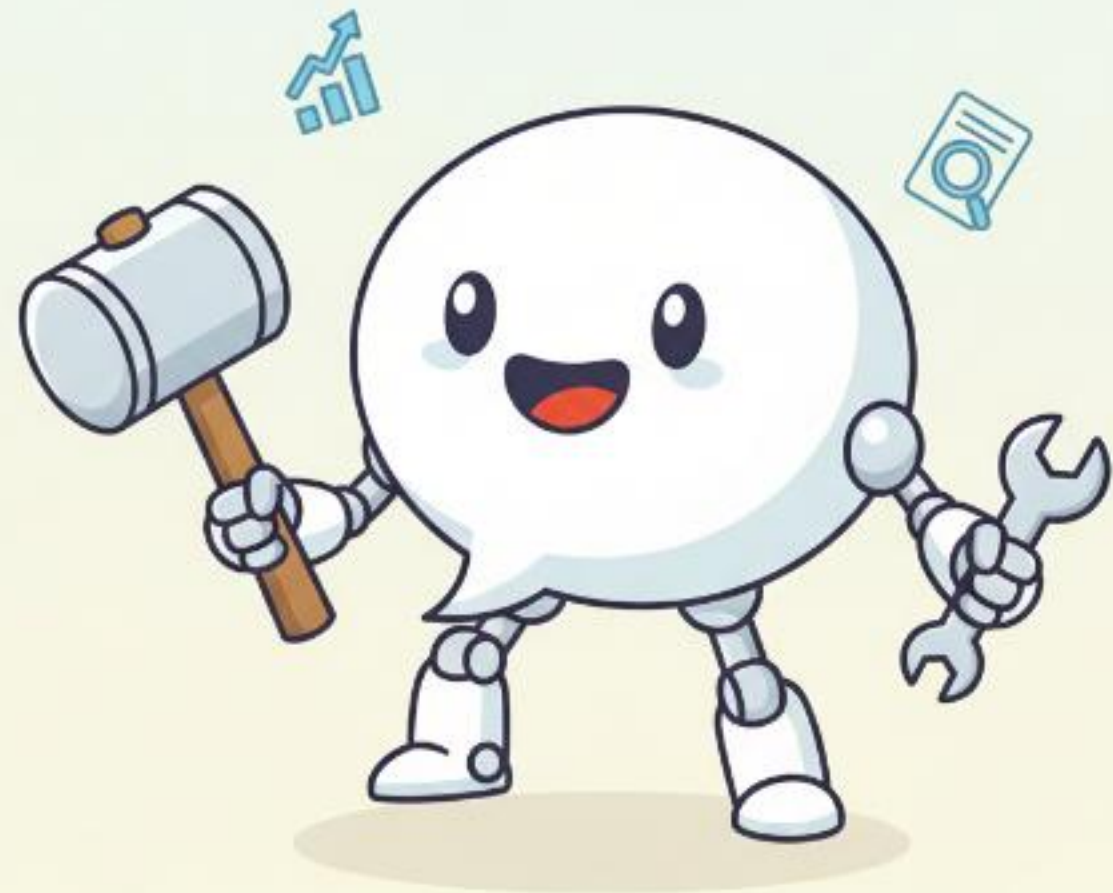
Tauhid Zaman

Chatbots Just Talk



Give The Chatbot Tools

- A **tool** is a **function your code exposes** to the AI model (plus a short schema: name, inputs, when to use it)
- The AI model only **requests** the call — *your software* runs it and returns the result
- This lets the chat bot do things: it can fetch live prices, search the web, etc.



What Is A Tool?

- A tool is **two pieces** that travel together:

1. The Function

Python you write and run — fetch a price, call an API, compute Sharpe...

2. The Schema

JSON the **AI model** reads — name, description, parameters — so it knows *when* and *how* to request the call

```
# 1) your function
def get_stock_price(ticker: str) -> dict: ...

# 2) the schema you register with the AI model (easy to mess up by hand)
{
  "type": "function",
  "name": "get_stock_price",
  "description": "Live last price for a US ticker.",
  "parameters": {
    "type": "object",
    "properties": {
      "ticker": { "type": "string" }
    },
    "required": ["ticker"]
  }
}
```

- Schemas are fiddly (names, types, nesting). That's why we **vibe-code** them instead of typing by hand

How A Tool Call Works



1 · ASK

"What is Apple's stock price?"



2 · PICK A TOOL

Reads the tool menu & chooses one



3 · REQUEST

Sends
`get_stock_price("AAPL")` to you



4 · YOU RUN

Your machine runs Python & sends the result back



5 · ANSWER

Replies in plain text: "AAPL is at \$187..."

TOOL MENU

get_stock_price

Live last price for a US ticker

compute_volatility

Recent realized volatility

web_search

Live web news (hosted)

PICKS

You Don't Write Every Tool

- OpenAI also has its **own tools** you can enable — see the [tools docs](#)

Web Search

Live web results + citations

`type: web_search`



Code Interpreter

Sandboxed Python on OpenAI's servers

`type: code_interpreter`



File Search

Search docs you uploaded

`type: file_search`



Computer

Control a desktop / UI with clicks & typing

`type: computer`



Image Generation

Create or edit images in the conversation

`type: image_generation`



Remote MCP

Plug in an external MCP server's tools

`type: mcp`



```
# add to the tools list – built-in + your function
tools=[ {"type": "web_search"}, {"type": "function", "name": "get_stock_price", ...} ]
```

Frontier Defaults Compared

Capability	OpenAI	Anthropic	Google	xAI
Custom functions	✓	✓	✓	✓
Code execution	✓	✓	✓	✓
Web search	✓	✓	✓	✓
File search	✓	✓	✓	✓
Computer	✓	✓	✓	—
Distinctive	Image gen tool; remote MCP	Skills API	Maps grounding	X Search (Twitter)

- Default toolboxes are converging — your edge is **custom tools** + how you use them



Tool Cost

OpenAI tool fees (plus model tokens either way)

Tool	OpenAI price
Web search	~\$0.01 / call + search tokens
Code Interpreter	from ~\$0.03 / 20-min container
File search	\$0.0025 / call + \$0.10/GB-day for file storage
Image gen	~270–4k tokens / image
Custom function	\$0 tool fee

- **Hosted:** vendor runs it — pay tool fees; best for hard commodity capabilities (web search, code interpreter, image models)
- **Custom:** you run it — \$0 OpenAI tool fee; best for your data and high-volume simple actions



Skills



| What Is A Skill?

- A **skill** is a short *playbook* (recipe) your coding agent follows for one kind of job
- It lives in a folder. The main file is always `SKILL.md`
- Write it once → reuse it whenever that job comes up
- **Tools** = the appliances (blender, oven) — what the AI can *call*
- **Skills** = the recipe — *how* you want the job done



What Goes In SKILL.md

1 When to use it

Which asks should load this playbook

2 Steps

What good looks like, in order

3 House rules

Naming, packages, do / don't

4 A tiny example

So the agent copies your style

Example · `.codex/skills/stock-chatbot/SKILL.md`

```
---
name: stock-chatbot
description: Build a Dash stock research chatbot
  with live yfinance tools. Use when asked for
  a stock chatbot or Dash + tool calling.
---

# Stock Research Chatbot

1. Dash chat UI – modern black + pink look.
2. OpenAI via Portkey (.env API key);
  model gpt-5.6-luna.
3. Market data with yfinance – never invent prices.
4. Tools: get_price, get_momentum,
  get_volatility, get_sharpe.
5. Optional: hosted web_search for news.
6. Keep tool returns short.
```


SKILL.md Vs AGENTS.md

- **SKILL.md** = specialist playbook for one job (called when needed)
- **AGENTS.md** = always-on handbook for the whole project (Codex loads it automatically — keep under ~10 pages)
- One project: **one AGENTS.md**, but you can have **many** skills

SKILL.md · skill folder (only when needed)

```
---
name: stock-chatbot
description: Build Dash stock chatbots with
  yfinance tools. Use when asked for a stock
  research chatbot.
---
```

Stock Research Chatbot

1. Dash UI – modern black + pink
2. Portkey + gpt-5.6-luna
3. Live yfinance price / vol / Sharpe
4. Never invent prices

AGENTS.md · project root (always on)

```
# MGT 409 student project
```

- Put PORTKEY_API_KEY in .env (never commit secrets)
- Default chat model: gpt-5.6-luna
- UI style: modern black + pink
- Draw Labubus with HTML/CSS/SVG (no image files)
- PDF text: use pypdf
- Prefer Dash or simple HTML apps you can demo

Making A Skill Is Easy

- In Codex chat, just ask — the agent writes the folder + `SKILL.md`

What you type

Add a skill for building Dash stock research chatbots with yfinance tools.

Use Portkey + gpt-5.6-luna. Modern black + pink UI. Live data for price, momentum, vol, and Sharpe – never invent prices. Use it whenever I ask for a stock chatbot.

What you get

```
.codex/skills/stock-chatbot/  
└─ SKILL.md
```

The agent fills in the playbook (when to use it + steps + house rules).

Next time you say “build the stock chatbot,” it loads that skill.

Skills You Might Write

stock-chatbot

Build a Plotly Dash stock research chat app with live Yahoo Finance tools, Portkey, and a modern black-and-pink look — never invent prices

case-memo

Turn a business-school case into a one-page memo: recommendation first, then 3 reasons, cite the exhibits, crisp language

pdf-to-tables

Pull text from homework or statement PDFs with pypdf, clean headers and dates, export tidy tables or a spreadsheet file

comps-sheet

Investment banking peer comps: pull peers from filings, keep units consistent, note every source, flag outliers

client-deck

Consulting slide story: one point per slide, no overlapping buckets, end with clear next steps the client can act on

grant-draft

Nonprofit grant narrative: need, program, budget, and measurable impact — match the funder's language

Tools + Skills Together

- Write an `AGENTS.md` for the whole project (always-on house rules)
- Add the **skills** this build needs (specialist playbooks)
- Use skills to help vibe-code an app that uses tools
- [Open the vibe coding page](#) →

